

TP1 -- Introduction a la compilation et premiers pas en C

Matiere : Decouverte des langages compiles

Niveau : Bachelor 1

Duree : 4h (cours + atelier)

Jour : 1/5 — lundi 11 mai 2026

Objectifs pedagogiques couverts :

- Comprendre la difference entre langages compiles et interpretes
- Citer les grands cas d'usage de C et de C++
- Installer et verifier GCC sur sa machine
- Ecrire, compiler et executer un programme C minimal
- Comprendre ce que font printf, return et le code de sortie

Contexte

Quand vous ecrivez un programme en Python, vous pouvez l'executer directement : l'interpreteur lit ligne par ligne et execute au fur et a mesure. En C, c'est different : votre code source doit d'abord etre **transforme en langage machine** par un compilateur avant de pouvoir etre execute. Ce TP vous guide pas a pas a travers vos premiers programmes compiles.

Prerequis logiciels

- GCC (GNU Compiler Collection) installe
 - Linux : `sudo apt install build-essential`
 - macOS : `xcode-select --install`
 - Windows : installer MSYS2 puis `pacman -S mingw-w64-x86_64-gcc`
- Un editeur de texte (VS Code recommande, ou Notepad++, vim...)
- Un terminal (Bash, PowerShell, ou Git Bash)

Partie 1 -- Compile vs Interprete (45 min)

1.1 Tableau comparatif a completer

Remplissez le tableau suivant en vous basant sur le cours et vos recherches :

Critere	Langage compile (C, C++, Rust)	Langage interprete (Python, JS)
Quand la traduction se fait-elle ?		
Vitesse d'execution		
Portabilite du binaire		
Detection des erreurs		
Besoin d'un runtime installe ?		
Exemples de langages		

1.2 Questions de reflexion

1. Pourquoi un programme C est-il generalement plus rapide qu'un programme Python equivalent ?
2. Que se passe-t-il si vous compilez un programme C sur Windows et essayez d'executer le fichier resultant sur Linux ?
3. Les interpreteurs Python, Ruby et JavaScript sont eux-memes ecrits en C/C++. Que cela nous dit-il sur l'importance des langages compiles ?
4. Donnez deux exemples concrets d'applications ou la performance impose l'usage d'un langage compile (pensez aux domaines vus en cours : systemes, embarque, jeux...).

Partie 2 -- Installation et verification de GCC (15 min)

2.1 Verifier l'installation

```
gcc --version
```

Si la commande fonctionne, notez la version de GCC affichee.

Question : Quelle version de GCC avez-vous ? Sur quel systeme d'exploitation etes-vous ?

2.2 Creer votre espace de travail

```
mkdir -p langages_compiles/jour1  
cd langages_compiles/jour1
```

Bonne pratique : Sauvegardez vos fichiers dans un dossier dedie au cours. Nous y reviendrons tous les jours.

Partie 3 -- Votre premier programme C (1h)

3.1 Anatomie d'un programme C

```
#include <stdio.h> // Inclusion de la bibliotheque standard

int main(void) {    // Fonction principale : point d'entree d
    printf("Bonjour le monde compile !\n"); // Affichage dans
    return 0;       // Code de retour : 0 = succes
}
```

Element	Role
<code>#include <stdio.h></code>	Inclut les fonctions d'entree/sortie (printf, scanf...)
<code>int main(void)</code>	Point d'entree obligatoire du programme
<code>printf(...)</code>	Affiche du texte dans le terminal
<code>return 0</code>	Indique au systeme que le programme s'est termine sans erreur
<code>\n</code>	Caractere de saut de ligne

3.2 Compiler et exécuter

```
gcc hello.c -o hello
./hello
```

Decortiquons la commande :

- `gcc` : le compilateur GNU
- `hello.c` : le fichier source à compiler
- `-o hello` : le nom de l'exécutable produit

3.3 Le code de sortie

Après exécution, le système reçoit un **code de sortie** :

```
./hello
echo $?    # Affiche 0 (succes)
```

3.4 Exercices progressifs

Exercice A -- Hello World

Creez `hello.c` avec le code ci-dessus, compilez-le et exécutez-le. Vérifiez le code de sortie avec `echo $?` .

Exercice B -- Affichage personnalisé

Creez `presentation.c` qui affiche les lignes suivantes (avec VOS informations) :

```
=== Mon premier programme C ===
Prenom : Alice
Niveau : Bachelor 1
Ecole  : YNOV Montpellier
```

Indice : Utilisez plusieurs appels a `printf` ou un seul avec plusieurs `\n` .

Exercice C -- Variables et affichage

Creez `variables.c` qui :

1. Declare une variable `age` de type `int` avec votre age
2. Declare une variable `taille` de type `float` avec votre taille en metres
3. Affiche : `"J'ai 20 ans et je mesure 1.75 m"`

```
#include <stdio.h>

int main(void) {
    int age = 20;
    float taille = 1.75f;

    // Completez l'affichage ici
    // %d pour les entiers, %.2f pour les flottants (2 decimal
    printf("J'ai %d ans et je mesure %.2f m\n", age, taille);

    return 0;
}
```

Exercice D -- Code de sortie personnalise

Creez `code_sortie.c` qui retourne un code de sortie **different de 0** (par exemple `return 42;`). Compilez, executez, et verifiez avec `echo $?` .
Que signifie un code de sortie different de 0 en convention Unix ?

Partie 4 -- Identifier et corriger des erreurs (1h)

4.1 Types d'erreurs en C

Type	Moment de detection	Exemple
Erreur de syntaxe	Compilation	Oubli de ;
Warning	Compilation	Variable non utilisee
Erreur a l'execution	Runtime	Division par zero

4.2 Lire un message d'erreur GCC

Compilez intentionnellement un programme avec une erreur et observez :

```
main.c:5:14: error: expected ';' before 'return'
```

Decomposez ce message :

- `main.c` : nom du fichier
- `5` : numero de ligne
- `14` : numero de colonne
- `error` : niveau de gravite
- `expected ';' before 'return'` : description du probleme

Regle d'or : Lisez toujours le PREMIER message d'erreur en priorite.
Les suivants sont souvent des consequences en cascade.

4.3 Debogage : corrigez ces programmes

Pour chaque programme, identifiez l'erreur, expliquez-la, puis corrigez-la.

Programme 1 -- Erreur de syntaxe :

```
#include <stdio.h>

int main(void) {
    printf("Bonjour le monde!")
    return 0;
}
```

Programme 2 -- Mauvais formats :

```
#include <stdio.h>

int main(void) {
    int nombre = 42
    float pi = 3.14;
    printf("Nombre : %f, Pi : %d\n", nombre, pi);
    return 0;
}
```

Il y a **deux** erreurs : une de syntaxe et une logique (formats `%d` et `%f` inverses).

Programme 3 -- Bug a l'execution :

```
#include <stdio.h>

int main(void) {
    int a = 10;
    int b = 0;
    int resultat = a / b;
    printf("Resultat : %d\n", resultat);
    return 0;
}
```


Celui-ci compile mais plante a l'execution. Pourquoi ? Comment le corriger ?

Programme 4 -- Bug logique :

```
#include <stdio.h>

int main(void) {
    int score = 85;
    if (score = 100) {
        printf("Score parfait !\n");
    }
    return 0;
}
```

Ce programme compile et s'exécute sans planter. Pourtant il affiche "Score parfait !" alors que le score est 85. Trouvez le bug.

4.4 Compiler avec les warnings

Recompilez les programmes ci-dessus avec les drapeaux d'avertissement :

```
gcc -Wall -Wextra programme.c -o programme
```

Question : Quels warnings supplémentaires GCC détecte-t-il ? Pourquoi est-il recommandé de TOUJOURS compiler avec `-Wall -Wextra` ?

Partie 5 -- Mini-projet : ma fiche d'identite (30 min)

Creez `fiche.c` : un programme qui :

1. Declare des variables pour votre prenom (tableau de char), age (int), taille (float) et initiale (char)
2. Affiche une fiche d'identite formatee :

```
+-----+
|  FICHE D'IDENTITE  |
+-----+
| Prenom  : Alice    |
| Age     : 20 ans   |
| Taille  : 1.75 m   |
| Initiale: A        |
+-----+
| Code de sortie : 0 (succes) |
+-----+
```

Squelette :

```
#include <stdio.h>

int main(void) {
    char prenom[] = "Alice";
    int age = 20;
    float taille = 1.75f;
    char initiale = 'A';

    // Completez l'affichage formate ici
    // %s pour les chaines, %d pour les entiers
    // %c pour un caractere, %.2f pour les flottants

    return 0;
}
```

Defi bonus : Compilez avec `-Wall -Wextra` et corrigez tous les warnings.

Pour aller plus loin

- Commande `man gcc` dans le terminal pour decouvrir les options
- Testez `gcc -v hello.c` pour voir toutes les etapes en detail
- Outil en ligne [Compiler Explorer \(godbolt.org\)](https://godbolt.org) : visualise l'assembleur genere en temps reel
- Essayez de compiler le meme programme en C++ avec `g++` : que change-t-il ?